

GUÍA PASO A PASO

Clase 5

Tu app en un servidor: Gradio y Streamlit en Render

Material de apoyo para la Actividad de la Clase 5 · Unidad 2 · Aplicaciones desplegadas

Esta guía tiene **todo lo que necesitás para resolver la Actividad de la Clase 5**: los archivos, los comandos y la configuración de Render, en el orden en que se usan. Si en clase viste el deploy como demostración, acá está el procedimiento completo para hacerlo vos.

AL TERMINAR TENÉS QUE TENER

- ☐ Tu app de Gradio andando en Render, con su link.
- ☐ Una versión mínima en Streamlit, también en Render, con su link.
- ☐ La carpeta `Clase_05` de tu repo con un README que tenga los dos links.
- ☐ El README general de la raíz actualizado con la Clase 05.

La primera vez puede llevarte alrededor de una hora, contando las esperas: cada deploy tarda unos minutos en construirse. **La entrega es el link a tu repositorio de GitHub**, en la Tarea del aula virtual, hasta el viernes 25 de septiembre a las 23:59.

1. Organizá la carpeta Clase_05

Son dos aplicaciones distintas, así que van en dos carpetas. Render va a publicar cada una como un servicio aparte, y a cada servicio le vas a decir en qué carpeta está su app.

```
estructura del repositorio
Clase_05/
├── README.md
├── gradio/
│   ├── app.py
│   └── requirements.txt
└── streamlit/
    ├── app.py
    └── requirements.txt
```

`requirements.txt` es la lista de librerías que tu app necesita. En tu compu ya las tenés instaladas; el servidor arranca vacío y **instala solo lo que diga ese archivo**. Si una librería no está ahí, en el servidor no existe.

2. La app de Gradio, lista para la nube

Usá el Blocks que armaste en la Clase 4. Lo único que cambia es la última línea: el `launch`. Si tenías `share=True`, sacalo.

```

Clase_05/gradio/app.py (ejemplo mínimo)
import os
import gradio as gr

def saludar(nombre):
    return f"Hola {nombre}"

with gr.Blocks() as demo:
    nombre = gr.Textbox(label="Tu nombre")
    salida = gr.Textbox(label="Respuesta")
    boton = gr.Button("Saludar")
    boton.click(fn=saludar, inputs=nombre, outputs=salida)

demo.launch(
    server_name="0.0.0.0",
    server_port=int(os.environ.get("PORT", 7860))
)

```

```

Clase_05/gradio/requirements.txt
gradio

```

Parte	Qué hace
<code>server_name="0.0.0.0"</code>	Atiende pedidos que vengan de cualquier lado. Con localhost solo se escucha a sí misma y Render no la encuentra.
<code>os.environ.get("PORT", 7860)</code>	Usa el puerto que le indica Render (por defecto, el 10000). En tu compu no existe esa variable, así que usa el 7860.
<code>int(...)</code>	El puerto tiene que ser un número; lo que viene del entorno es texto.

Probalo en tu compu antes de subirlo, parado en la carpeta [Clase_05/gradio](#):

```

terminal
pip install -r requirements.txt
python app.py

```

Abri <http://localhost:7860> en el navegador. Si anda acá, el código está bien; lo que falle después va a ser de configuración.

3. La versión mínima en Streamlit

No tiene que ser igual de completa que la de Gradio: alcanza con la misma función y una interfaz simple. La idea es comprobar que el procedimiento de deploy es el mismo.

```

Clase_05/streamlit/app.py
import streamlit as st

def saludar(nombre):
    return f"Hola {nombre}"

st.title("Saludador")
nombre = st.text_input("Tu nombre")

if st.button("Saludar"):
    st.write(saludar(nombre))

```

```
Clase_05/streamlit/requirements.txt
streamlit
```

```
terminal, parado en Clase_05/streamlit
pip install -r requirements.txt
streamlit run app.py
```

Se abre sola en <http://localhost:8501>. Fijate que acá no hay `launch`: a Streamlit la arranca el comando `streamlit run`, y el puerto se le pasa en ese comando (paso 6).

UNA DIFERENCIA PARA TU README

En Gradio, cada botón llama a una función. En Streamlit, cada vez que tocás algo **se vuelve a ejecutar todo el script de arriba abajo**. Es la diferencia principal entre las dos, y un buen tema para las 2 o 3 líneas de comparación que pide la Actividad.

4. Subí todo a GitHub

```
terminal, en la raíz de tu repositorio
git add Clase_05
git commit -m "Clase 05: apps de Gradio y Streamlit para Render"
git push origin main
```

Entrá a tu repo en GitHub y abrí las carpetas `gradio` y `streamlit`: tienen que verse los archivos. Si ves una carpeta gris que no abre, es el `.git` anidado de la clase: está resuelto en el Machete de la Clase 5, sección 4.

5. Creá el servicio de Gradio en Render

1. Entrá a render.com y registrate con tu cuenta de GitHub. El plan gratuito no pide tarjeta.
2. Arriba: **New** → **Web Service**.
3. Elegí tu repositorio. Si no aparece, apretá **Configure account** (o el link para ajustar permisos de GitHub) y dale acceso a Render a ese repo.
4. Completá el formulario con la tabla de abajo y apretá **Deploy Web Service**.

Campo en Render	Qué poner
Name	Algo como <code>apellido-gradio</code> . Va a ser parte del link.
Language	Python 3
Branch	<code>main</code>
Root Directory	<code>Clase_05/gradio</code> — la carpeta donde está esta app. Es el campo que más se olvida.
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>python app.py</code>
Instance Type	Free

Se abre la pantalla de logs. Leelos mientras construye: primero instala las librerías y después arranca la app. Cuando termina aparece **Your service is live** y arriba, el link <https://<nombre>.onrender.com>.

6. Creá el servicio de Streamlit

Otra vez **New** → **Web Service**, mismo repositorio. Cambian tres campos:

Campo en Render	Qué poner
Name	apellido-streamlit
Root Directory	Clase_05/streamlit
Start Command	streamlit run app.py --server.port \$PORT --server.address 0.0.0.0 --server.headless true

El resto, igual que en el paso 5. Compará el Start Command con el **launch** de Gradio: **es el mismo ajuste escrito en otro lugar**.

Gradio (en el código)	Streamlit (en el comando)	Para qué
server_name="0.0.0.0"	--server.address 0.0.0.0	Aceptar pedidos de afuera
server_port=...PORT...	--server.port \$PORT	Escuchar donde Render espera
—	--server.headless true	No intentar abrir un navegador en el servidor

7. Verificá que viva sin tu computadora

1. Abrí los dos links. Si anda, **cerrá tu compu y probalos desde el celular**: esa es la diferencia con el link de share=True.
2. El plan gratuito **duerme el servicio después de 15 minutos sin visitas**. La primera visita después tarda cerca de un minuto en despertarlo. No es un error.
3. Cada **git push** a main vuelve a publicar la app sola. Si no se actualizó, en Render: **Manual Deploy** → **Deploy latest commit**.

8. Si algo falla, mirá los logs

Antes de preguntarle a una IA, leé el mensaje de error completo en los logs de Render: casi siempre dice qué pasó.

Lo que ves en los logs	Por qué pasa	Qué hacer
No open ports detected	La app escucha en localhost o en un puerto fijo.	Revisá el launch (Gradio) o los flags del Start Command (Streamlit).
Could not open requirements file	Render está parado en otra carpeta.	Revisá el Root Directory (Settings → Build & Deploy).
ModuleNotFoundError: No module named ...	Falta esa librería en requirements.txt.	Agregala, commit y push.
Tu repo no aparece al crear el servicio	Render no tiene permiso sobre ese repo.	Ajustá los permisos de GitHub desde Render.

Lo que ves en los logs	Por qué pasa	Qué hacer
El link tarda mucho la primera vez	El servicio estaba dormido.	Esperá un minuto y recargá.

SI TE TRABÁS

Escribí en el foro del Ticket de salida de la Clase 5 qué paso estabas haciendo y **copiá el mensaje de error de los logs**. Seguro no sos la única persona con ese problema.

9. Documentá en el README de Clase_05

Un ejemplo para arrancar. Cambialo como quieras: tiene que entenderlo alguien que entra por primera vez a tu repo.

```

Clase_05/README.md
# Clase 05 – Mi app en un servidor

## Links
- Gradio en Render: https://apellido-gradio.onrender.com
- Streamlit en Render: https://apellido-streamlit.onrender.com

## Qué hace cada una
...

## Cómo se publicó
Render, Web Service gratuito. Root Directory, Build y Start Command de cada
una.

## Qué me resultó distinto entre Gradio y Streamlit
2 o 3 líneas.
```

Y en el README general de la raíz, sumá la Clase 05 al índice. Después: commit, push, y pegá el link del repositorio en la Tarea.

Si estás en una computadora sin permisos para instalar

Render funciona entero desde el navegador. Los archivos los podés crear en GitHub: **Add file → Create new file** y en el nombre escribís la ruta completa, por ejemplo `Clase_05/gradio/app.py`; las carpetas se crean solas. Lo único que no vas a poder es probar en tu compu: probás directo en Render, leyendo los logs.

Documentación oficial

1. **Render — Web Services:** por qué hay que escuchar en 0.0.0.0 y en el puerto PORT. render.com/docs/web-services
2. **Render — plan gratuito:** límites y por qué el servicio se duerme. render.com/docs/free
3. **Streamlit — primeros pasos:** cómo se escribe una interfaz. docs.streamlit.io